САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №3

Выполнил:

Федоров Даниил

Группа

K3440

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

- реализовать Dockerfile для каждого сервиса;
- написать общий docker-compose.yml;
- настроить сетевое взаимодействие между сервисами.

Ход работы

Для ЛР3 был создан отдельный Docker Compose-проект job-search-lr3. Чобы исключить конфликты с контейнерами предыдущих работ, для публикации сервисов были выбраны отдельные порты. Внутри Docker контейнеров используются стандартные порты сервисов, а на хость машине им соотвествующие отдельные порты.

docker-compose.yml

```yaml
name: job-search-lr3
# ▷Run All Services
services:
  # ▷Run Service
  postgres:
    image: postgres:16
    restart: always

    ports:
      - 16432:5432

    environment:
      POSTGRES_USER: maindb
      POSTGRES_PASSWORD: maindb
      POSTGRES_DB: maindb

    volumes:
      - postgres-data:/var/lib/postgresql/data
      - ./docker/init-databases.sql:/docker-entrypoint-initdb.d/init-databases.sql



  # ▷Run Service
  rabbitmq:
    image: rabbitmq:4-management
    restart: always

    hostname: rabbitmq

    ports:
      - "5673:5672"
      - "15673:15672"

    environment:
      RABBITMQ_DEFAULT_USER: app
      RABBITMQ_DEFAULT_PASS: app

    volumes:
      - rabbitmq-data:/var/lib/rabbitmq

volumes:
  postgres-data:
  rabbitmq-data:
```

Для хранения данных PostgreSQL используется именованный Docker volume. При первом запуске контейнера выполняется SQL скрипт инициализации, автоматически создающий отдельные базы данных для микросервисов.

```
docker > init-databases.sql
1    CREATE DATABASE user_db;
2    CREATE DATABASE reference_db;
3    CREATE DATABASE vacancy_db;
4    CREATE DATABASE response_db;
```

Проводим запус инфраструктуры. Создались два контейнера с БД и RabbitMQ.

```
fdanila@MacBook-Air-Danill-4 express-typeorm-microservices % docker compose ps
NAME                          IMAGE                  COMMAND                  SERVICE    CREATED          STATUS          PORTS
job-search-lr3-postgres-1     postgres:16            "docker-entrypoint.s…"   postgres   12 seconds ago   Up 12 seconds   0.0.0.0:16432->5432/tcp, [::]:16432->5432/tcp
job-search-lr3-rabbitmq-1     rabbitmq:4-management  "docker-entrypoint.s…"   rabbitmq   12 seconds ago   Up 12 seconds   0.0.0.0:5673->5672/tcp, [::]:5673->5672/tcp, 0.0.0.0:15673->15672/tcp, [::]:15673->15672/tcp
```

Далее перейдем к приложению внутри докер. Для User Service был создан отдельный Dockerfile на основе образа Node.js. Исходный код и зависимости приложения копируются внутрь контейнера, после чего сервис запускается командой npm run dev. И был добавлен dockerignore чтобы docker не копировал локальный node_modules внутрь linux контейнера.

```
user_service > Dockerfile > ...
1    # берем готовый linux образ с node.js
2    FROM node:20-slim
3    # устанавливаем рабочую директорию внутри контейнера
4    WORKDIR /app
5    # Копируем зависимости проекта в контейнер
6    COPY package*.json ./
7    #устанавливаем зависимости проекта
8    RUN npm install
9    # копируем все файлы проекта в контейнер
10   COPY . .
11   # приложение внутри контейнера работает на порту 8001
12   EXPOSE 8001
13   #Запускаем приложение внутри контейнера
14   CMD ["npm", "run", "dev"]
```

В Docker Compose для сервиса настроен внешний порт 8101 и внутренний порт 8001.При контейнеризации сетевые настройки были изменены. Для соединения с PostgreSQL вместо localhost используется имя Compose-сервиса db и внутренний порт 5432. Приложение запускается на адресе 0.0.0.0, что позволяет обращаться к нему через опубликованный порт контейнера.Так же добавляем условие, что сервис запускается только после запуска БД

```yaml
user_service:
  build:
    context: ./user_service

  restart: always

  ports:
    - 8101:8001

  environment:
    APP_HOST: 0.0.0.0
    APP_PORT: 8001
    APP_PROTOCOL: http
    APP_API_PREFIX: /api/v1

    DB_HOST: db
    DB_PORT: 5432
    DB_NAME: user_db
    DB_USER: maindb
    DB_PASSWORD: maindb

    JWT_SECRET_KEY: secret
    JWT_TOKEN_TYPE: Bearer
    JWT_ACCESS_TOKEN_LIFETIME: 3600

    REFERENCE_SERVICE_URL: http://reference_service:8002/api/v1
    INTERNAL_SERVICE_TOKEN: my-internal-service-token

  depends_on:
  #сервис user_service будет ждать пока сервис db не будет готов к работе
    db:
      condition: service_healthy
```

Запускаем сервис и проверяем

```
fdanila@MacBook-Air-Daniil-4 express-typeorm-microservices % docker compose ps
NAME                                IMAGE                          COMMAND                  SERVICE        CREATED          STATUS                    PORTS
job-search-lr3-db-1                 postgres:16                    "docker-entrypoint.s…"   db             27 seconds ago   Up 26 seconds (healthy)   0.0.0.0:16432->5432/tcp, [::]:16432->5432/tcp
job-search-lr3-rabbitmq-1           rabbitmq:4-management          "docker-entrypoint.s…"   rabbitmq       11 minutes ago   Up 11 minutes             0.0.0.0:5673->5672/tcp, [::]:5673->5672/tcp, 0.0.0
72/tcp
job-search-lr3-user-service-1   job-search-lr3-user-service   "docker-entrypoint.s…"   user-service   27 seconds ago   Up 20 seconds             0.0.0.0:8101->8001/tcp, [::]:8101->8001/tcp
fdanila@MacBook-Air-Daniil-4 express-typeorm-microservices % docker compose logs -f user-service
user-service-1  |
user-service-1  | > express-typeorm-boilerplate@1.0.0 dev
user-service-1  | > node --env-file=.env --watch -r ts-node/register src/app.ts
user-service-1  |
user-service-1  | User Service running on http://0.0.0.0:8001
user-service-1  | Swagger: http://0.0.0.0:8001/docs
user-service-1  | User Service database has been initialized!
```

Внутри других сервисов создаем так же докерфайлы и dockerignore.

Добавляем reference service

```yaml
reference-service:
  build:
    context: ./reference_service

  restart: always

  ports:
    - 8102:8002

  environment:
    APP_HOST: 0.0.0.0
    APP_PORT: 8002
    APP_PROTOCOL: http
    APP_API_PREFIX: /api/v1

    DB_HOST: db
    DB_PORT: 5432
    DB_NAME: reference_db
    DB_USER: maindb
    DB_PASSWORD: maindb

    INTERNAL_SERVICE_TOKEN: my-internal-service-token

  depends_on:
    db:
      condition: service_healthy
```

Добавляем vacancy-service

```
▷Run Service
vacancy-service:
  build:
    context: ./vacancy_service

  restart: always

  ports:
    - "8103:8003"

  environment:
    APP_HOST: 0.0.0.0
    APP_PORT: 8003
    APP_PROTOCOL: http
    APP_API_PREFIX: /api/v1

    DB_HOST: db
    DB_PORT: 5432
    DB_NAME: vacancy_db
    DB_USER: maindb
    DB_PASSWORD: maindb

    JWT_SECRET_KEY: secret
    JWT_TOKEN_TYPE: Bearer
    JWT_ACCESS_TOKEN_LIFETIME: 3600

    USER_SERVICE_URL: http://user-service:8001/api/v1
    REFERENCE_SERVICE_URL: http://reference-service:8002/api/v1

    INTERNAL_SERVICE_TOKEN: my-internal-service-token

    RABBITMQ_URL: amqp://app:app@rabbitmq:5672

  depends_on:
    db:
      condition: service_healthy
    user-service:
      condition: service_started
    reference-service:
      condition: service_started
    rabbitmq:
      condition: service_started
```

Добавляем response service

```
▷Run Service
response-service:
  build:
    context: ./response_service

  restart: always

  ports:
    - "8104:8004"

  environment:
    APP_HOST: 0.0.0.0
    APP_PORT: 8004
    APP_PROTOCOL: http
    APP_API_PREFIX: /api/v1

    DB_HOST: db
    DB_PORT: 5432
    DB_NAME: response_db
    DB_USER: maindb
    DB_PASSWORD: maindb

    JWT_SECRET_KEY: secret
    JWT_TOKEN_TYPE: Bearer
    JWT_ACCESS_TOKEN_LIFETIME: 3600

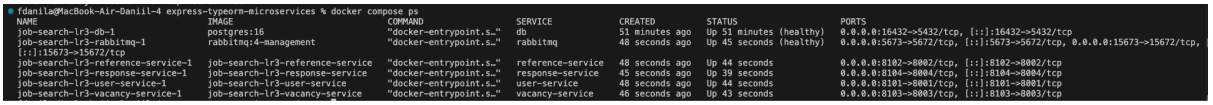
    USER_SERVICE_URL: http://user-service:8001/api/v1
    VACANCY_SERVICE_URL: http://vacancy-service:8003/api/v1

    INTERNAL_SERVICE_TOKEN: my-internal-service-token

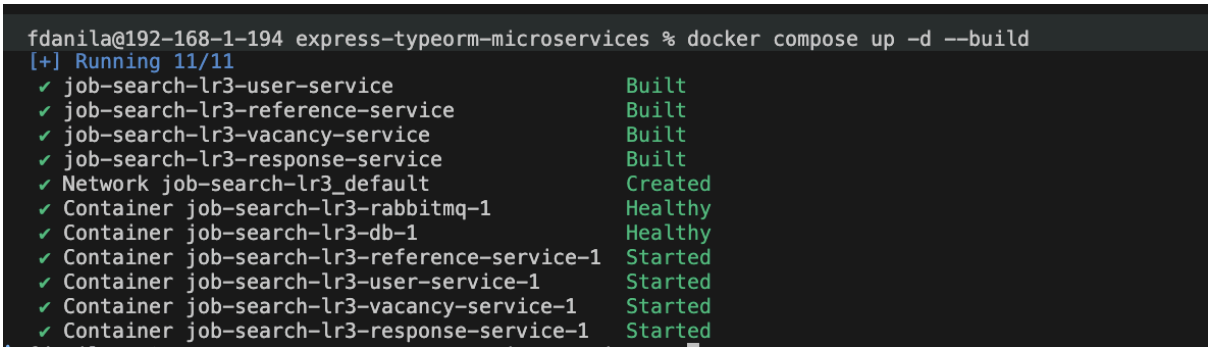
    RABBITMQ_URL: amqp://app:app@rabbitmq:5672

  depends_on:
    db:
      condition: service_healthy
    rabbitmq:
      condition: service_healthy
    user-service:
      condition: service_started
    vacancy-service:
      condition: service_started
```

Теперь мы можем увидеть все 6 контейнеров



Теперь проверяем сможем ли мы поднять всю систему одной командой docker compose up -d –build



все 6 контейнеров работают



Вывод

В ходе лабораторной работы микросервисное приложение было полностью контейнеризировано с использованием Docker Compose. В контейнерах были запущены четыре микросервиса, PostgreSQL и RabbitMQ, настроено их взаимодействие через внутреннюю Docker-сеть. В результате вся система может быть запущена одной командой docker compose up.